Evaluation Metrics Explained (With Your Real Numbers)

The winning model is Random Forest. Its confusion matrix on the 6,000-row held-out test set (`reports/figures/confusion_matrix_random_forest.png`):

	predicted: no default	predicted: default
actual: no default	3,840 (true negative)	833 (false positive)
actual: default	520 (false negative)	807 (true positive)

Everything below is computed from these four numbers.

Why not just use accuracy?

```
accuracy = (true negatives + true positives) / total
         = (3840 + 807) / 6000 = 0.775
```

77.5% looks decent in isolation — until you remember 78% of the test set *doesn't* default. A model that predicted "no default" for every single customer, with zero intelligence, would score **77.9% accuracy** — almost identical. Accuracy alone can't tell you whether a model is actually any good at the thing that matters: catching defaulters. That's why this project reports precision, recall, and F1 instead of leading with accuracy.

Precision: "Of the people I flagged as risky, how many actually were?"

```
precision = true positives / (true positives + false positives)
          = 807 / (807 + 833) = 0.492
```

49% of the customers this model flagged as "high risk" actually defaulted. The other 51% were false alarms — customers who'd have paid fine but got flagged for manual review anyway. Low precision means more wasted review effort.

Recall: "Of the people who actually defaulted, how many did I catch?"

```
recall = true positives / (true positives + false negatives)
       = 807 / (807 + 520) = 0.608
```

61% of actual defaulters were correctly caught by the model. The other 39% (520 people) slipped through as false negatives — the model said "fine," but they defaulted anyway.

The precision/recall trade-off

These two pull against each other. You could catch every single defaulter (recall = 100%) by just flagging everyone as high risk — but then precision collapses, because you'd also be flagging every good customer. Or you could get near-perfect precision by only flagging the most obvious, extreme cases — but then recall collapses, because you'd miss most real defaulters who aren't extreme. Neither extreme is useful; the business needs both to some degree, which is exactly why a single blended metric is useful for picking a model.

F1-score: the balance between the two

```
f1 = 2 * (precision * recall) / (precision + recall)
  = 2 * (0.492 * 0.608) / (0.492 + 0.608) = 0.544
```

F1 is the harmonic mean of precision and recall — it only scores high when *both* are reasonably good, which is why it (not accuracy) was used as the `scoring` parameter in `GridSearchCV` and as the tie-breaker between the three models.

Why recall matters more than precision for this specific business problem

Worth saying explicitly in an interview, even though this project doesn't implement a custom cost-sensitive threshold: **a missed defaulter (false negative) usually costs a lender far more than a false alarm (false positive)**. Flagging a good customer for extra review wastes a few minutes of staff time; missing an actual defaulter can mean losing the entire outstanding balance. That asymmetry is a good follow-up point to raise proactively — "if I were taking this further, I'd tune the decision threshold below 0.5 to trade some precision for more recall, since that better matches the actual cost of each type of mistake."

Why `class_weight="balanced"` instead of resampling (SMOTE, undersampling, etc.)

`class_weight="balanced"` tells the model to penalize mistakes on the minority class (default=1) more heavily during training — roughly proportional to how rare that class is (78/22 ~= 3.5x more penalty per default-class mistake). It achieves a similar goal to oversampling the minority class or undersampling the majority class, but without duplicating rows or throwing away real data, and it's a single parameter rather than an extra preprocessing step. For a dataset of this size and this moderate (not extreme) imbalance, it's the simplest tool that works.